

Documentación Oficial de Jules

Módulo 1: Guías y Documentación General

Google Jules Agent Docs

Fuente: <https://jules.google/docs>

Tabla de Contenido

1. Getting started
2. Environment setup
3. Running Tasks with Jules
4. Scheduled Tasks
5. Suggested Tasks
6. Reviewing plans & giving feedback
7. Reviewing code changes
8. Managing tasks and repos
9. Repo view
10. Errors and failures
11. Limits and Plans
12. Feedback and support
13. FAQ
14. Contact

Getting started

Fuente: <https://jules.google/docs/>

Getting started

Jules is an experimental coding agent that helps you fix bugs, add documentation, and build new features. It integrates with GitHub, understands your codebase, and works **autonomously** — so you can move on while it handles the task.

This guide will walk you through setting up Jules and running your first task.

Login

Section titled "Login"

1. Visit jules.google.com
2. Sign in with your Google account.
3. Accept the privacy notice (one-time).

Connect GitHub

[Section titled "Connect GitHub"](#)

Jules needs access to your repositories in order to work.

1. Click **Connect to GitHub account**.
2. Complete the login flow.
3. Choose *all* or specific repos that you want to connect to Jules.
4. You will be redirected back to Jules. If not, try refreshing the page.

Once connected, you'll see a **repo selector** where you can select the repo you want Jules to work with, and a prompt input box.

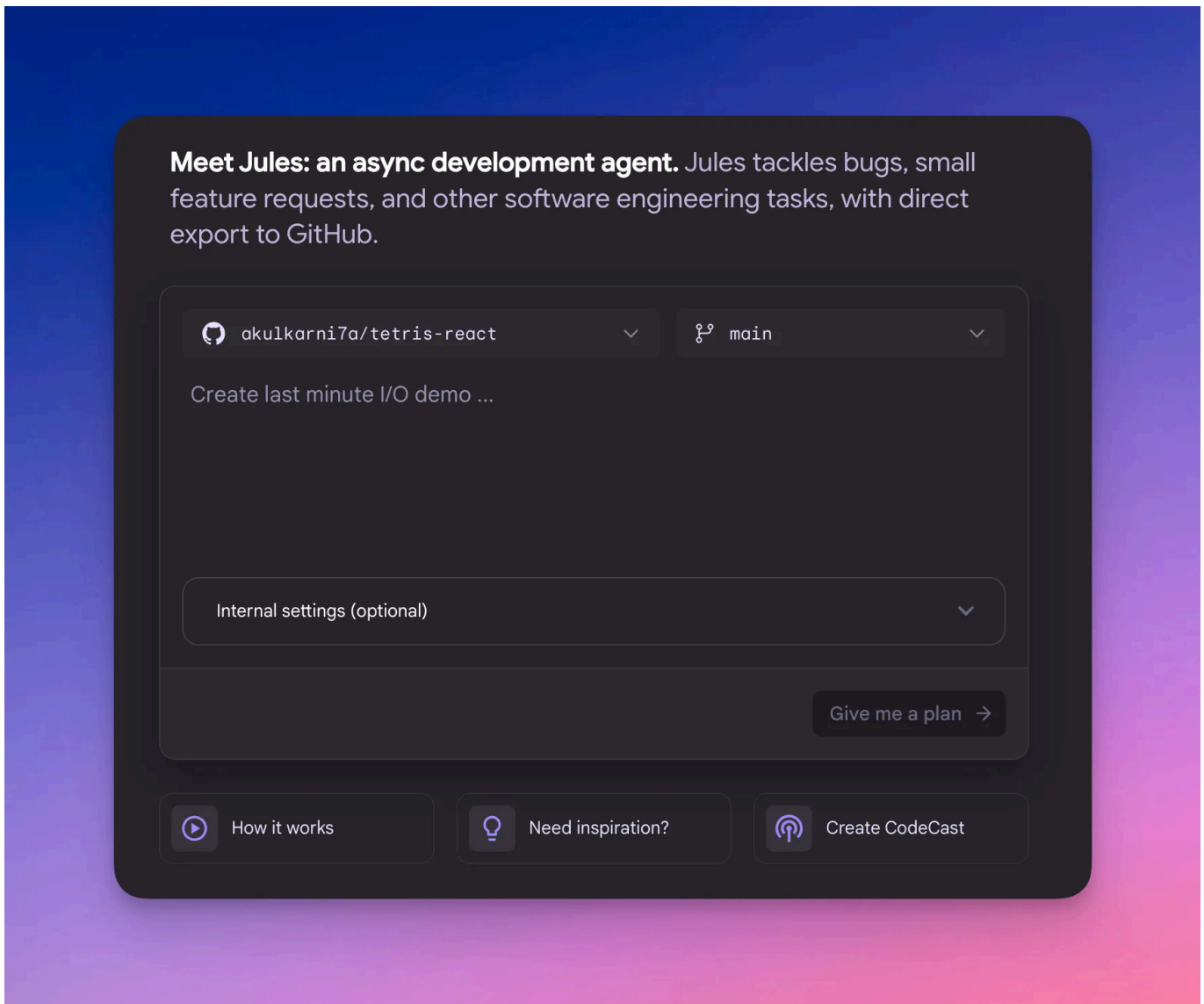
Starting your first task

[Section titled "Starting your first task"](#)

Jules runs in a virtual machine where it clones your code, installs dependencies, and modifies files.

1. Pick a repository from the repo selector.
2. Choose the branch you want Jules to work on. The default branch will be selected already. You do not have to modify this unless you want Jules to work on a specific branch.
3. Write a clear, specific prompt. For example, `Add a test for "parseQueryString" function in utils.js`
4. (Optional) Add environment setup scripts.
5. Click **Give me a plan**

Once you submit a task, Jules will generate a plan. You can review and approve it before any code changes are made.



Include AGENTS.md file

[Section titled "Include AGENTS.md file"](#)

Jules now automatically looks for a file named AGENTS.md in the root of your repository. This file can describe the agents or tools in your codebase, such as what they do, how to interact with them, or any input and output conventions. Jules uses this file to better understand your code and generate more relevant plans and completions.

Tip: Keep AGENTS.md up to date. It helps Jules and your teammates work with your repo more effectively.

Enabling notifications

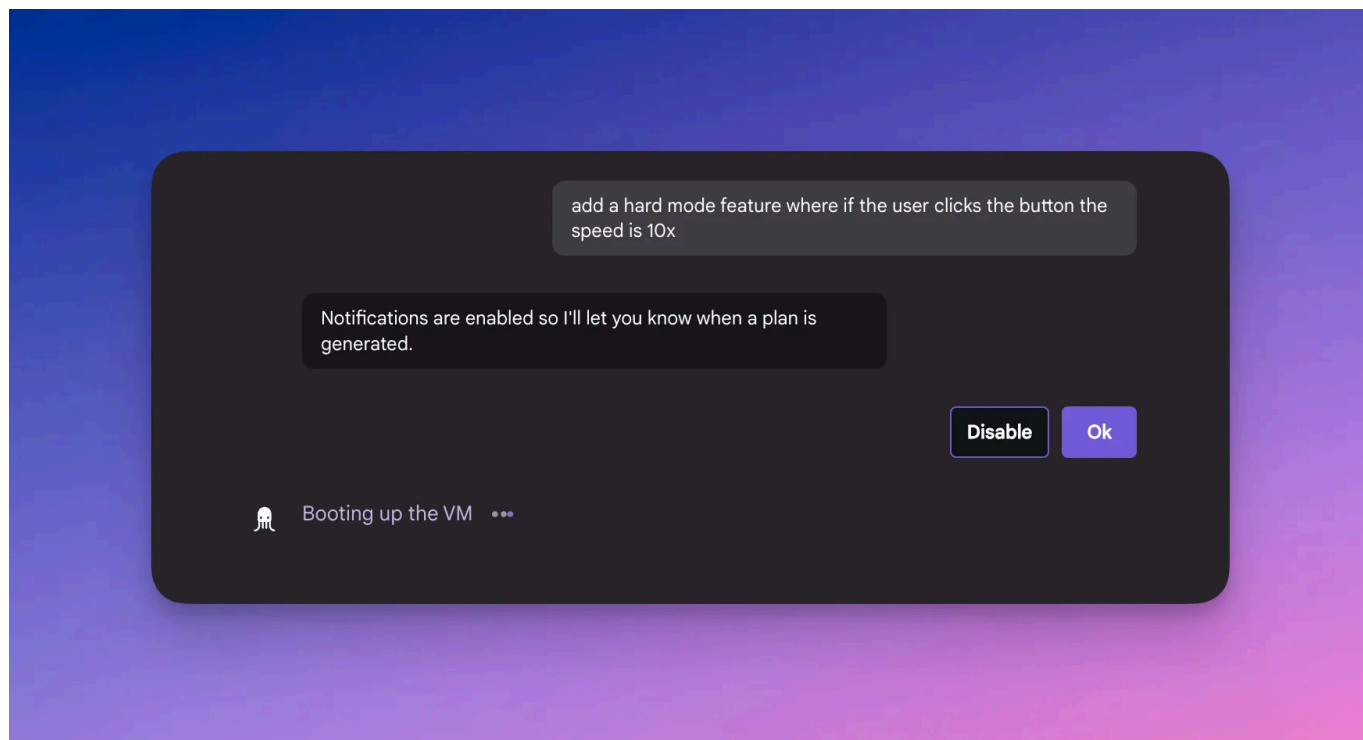
[Section titled "Enabling notifications"](#)

You are free to leave Jules while it is running. To stay informed:

1. When prompted, enable browser notifications.

2. Go to **Settings** —> **notifications** at any time to enable or disable notifications.

You'll be notified when the task completes or needs your input.



What's next?

Section titled "What's next?"

- [Running a task with Jules](#) - Full walkthrough
- [Environment setup](#) - Make Jules smarter about your project
- [Reviewing plans & feedback](#) - how to approve and integrates

Want to dive into real-world use cases? Check out the [Jules Awesome Prompts repo](#).

Environment setup

Fuente: <https://jules.google/docs/environment/>

Environment setup

Jules runs each task inside a secure, short-lived virtual machine (VM). This lets it clone your repository, install dependencies, and run tests.

For simple environments, Jules studies your repository to learn how to quickly setup your environment without a provided setup script. Jules will also refer to `agents.md` or your `readme.md` file for hints to setup an environment on the fly.

For complex environments, you can provide a **setup script** to be run explicitly that prepare the environment.

What's preinstalled?

[Section titled "What's preinstalled?"](#)

Every Jules VM runs Ubuntu Linux and includes many popular developer tools out of the box:

- Node.js
- Bun
- Python
- Go
- Java
- Rust

The latest versions can be seen here:

Environment check output

----- Python -----

- ✓ python3: Python 3.12.11
- ✓ python: Python 3.12.11
- ✓ pip: pip 25.1.1 from /home/jules/.pyenv/versions/3.12.11/lib/python3.12/site-packages/pip (python 3.12)
- ✓ pipx: 1.4.3
- ✓ poetry: Poetry (version 2.1.3)
- ✓ uv: uv 0.7.13
- ✓ black: black, 25.1.0 (compiled: yes)
- ✓ mypy: mypy 1.16.1 (compiled: yes)
- ✓ pytest: pytest 8.4.0
- ✓ ruff: ruff 0.12.0
- ✓ pyenv: available
 - system
 - 3.10.18
 - 3.12.11 (set by /home/jules/.pyenv/version)

----- NodeJS -----

- ✓ node: v22.16.0 *
 - v18.20.8 *
 - v20.19.2 *
 - v22.16.0 *
 - system *
- ✓ nvm: available
- ✓ npm: 11.4.2
- ✓ yarn: 1.22.22
- ✓ pnpm: 10.12.1
- ✓ eslint: v9.29.0
- ✓ prettier: 3.5.3
- ✓ chromedriver: ChromeDriver 137.0.7151.70
(dfa4dc56b2ahb56eb2a14cad006ea5e68c60d5de-refs/branch-heads/71510{#1875})

----- Java -----

- ✓ java: openjdk version "21.0.7" 2025-04-15
 - OpenJDK Runtime Environment (build 21.0.7+6-Ubuntu-0ubuntu124.04)
 - OpenJDK 64-Bit Server VM (build 21.0.7+6-Ubuntu-0ubuntu124.04, mixed mode, sharing)
- ✓ maven: Apache Maven 3.9.10 (5f519b97e9448438d878815739f519b2eade0a91d)
- ✓ gradle: Gradle 8.8

----- Go -----

- ✓ go: go version go1.24.3 linux/amd64

----- Rust -----

- ✓ rustc: rustc 1.87.0 (17067e9ac 2025-05-09)
- ✓ cargo: cargo 1.87.0 (99624be96 2025-05-06)

----- C/C++ Compilers -----

- ✓ clang: Ubuntu clang version 18.1.3 (1ubuntu1)
- ✓ gcc: gcc (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0
- ✓ cmake: cmake version 3.28.3
- ✓ ninja: 1.11.1
- ✓ conan: Conan version 2.17.0

----- Docker -----

- ✓ docker: Docker version 28.2.2, build e6534b4
- ✓ docker: Docker Compose version v2.36.2

----- Other Utilities -----

```
✓ awk: GNU Awk 5.2.1, API 3.2, PMA Avon 8-g1, (GNU MPFR 4.2.1, GNU MP 6.3.0)
✓ curl: curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotl
i/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/
zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.7
✓ git: git version 2.49.0
✓ grep: grep (GNU grep) 3.11
✓ gzip: gzip 1.12
✓ jq: jq-1.7
✓ make: GNU Make 4.3
✓ rg: ripgrep 14.1.0
✓ sed: sed (GNU sed) 4.9
✓ tar: tar (GNU tar) 1.35
✓ tmux: tmux 3.4
✓ yq: yq 0.0.0
```

You can check installed versions by adding commands like `node -v` to your setup script and clicking **Run to Validate**.

To view all preinstalled tools, you can use this command in your setup script and click **Run to Validate**.

```
set +x; . /opt/environment_summary.sh
```

Add a setup script

[Section titled "Add a setup script"](#)

To help Jules install dependencies and run tests:

1. Click on the repo on the left sidebar (under codebases)
2. Select Configuration at the top
3. In the "Initial Setup" window, enter the commands needed to install dependencies and prep your project

For example:



```
npm install
npm run test
```

Test your setup script

[Section titled "Test your setup script"](#)

Click **Run and Snapshot** to check that your setup script works. Upon success a snapshot of your environment will be created.

Environment Snapshots

[Section titled "Environment Snapshots"](#)

After you click **Run and Snapshot** your environment setup script will be run and you will see the results. Upon success, a snapshot of your environment is taken. This snapshot will be used for future Jules tasks started from this repository. This is especially useful for complex environments with long setup times.

Validation tips

Section titled "Validation tips"

- You can check installed versions by adding commands like `node -v` to your setup script and clicking **Run to Validate**.
- Always include commands to install packages, run linters, or execute tests.
- Use the validation button to catch errors early.
- Keep your setup lightweight and fast

Running Tasks with Jules

Fuente: <https://jules.google/docs/running-tasks/>

Running Tasks with Jules

Once you've logged in and connected GitHub, you're ready to start coding with Jules. This guide walks through the key steps of running a task — from selecting a repo to writing your prompt and setting up notifications.

Choose a repo and branch

[Section titled "Choose a repo and branch"](#)

Jules needs a repo and branch to work on. After logging in:

1. Open the **repo selector** dropdown.
2. Select the repository you'd like Jules to work on.
3. Choose the branch you want to base your changes on.

Jules remembers your last-used repo, so you'll always see the last used repo in the repo selector.

Write a clear prompt

[Section titled "Write a clear prompt"](#)

Jules works best when your prompt is specific and scoped. Use plain language — no need for perfect grammar or code.

Attach visual context

[Section titled "Attach visual context"](#)

You can upload images when initiating a task in Jules. Visuals like UI mockups, front-end glitches, screenshots, diagrams, or inspiration help clarify context when used with your natural language prompt. How to use it:

1. Drag-and-drop or browse to upload one or more images at task creation.
2. Total upload size must stay under 5MB.
3. Supported formats: PNG and JPEG.
4. Images are only attachable during initial task setup; not yet available in follow-up prompts.

Jules ingests these visuals to inform its understanding and output. They won't be embedded in code or committed to your repository. If you want visual assets included in code, upload them to your repo separately.

✓ Good prompts

- Add a loading spinner while `fetchUserProfile` runs
- Fix the 500 error while submitting the feedback form
- Document the `useCache hook with JSDoc`

⊘ Avoid

- Fix everything
- Optimize code
- Make this better

If Jules needs more clarity, it will ask for feedback before writing code.

Watching Jules work

[Section titled "Watching Jules work"](#)

Once the plan is approved, Jules will start coding.

You will see:

- An **activity feed** as each step completes
- Inline explanations of each change
- A **mini diff** preview for each file

Use the **diff editor** for a full view across all files.

Final summary & branch creation

[Section titled "Final summary & branch creation"](#)

When Jules finishes a task, it provides a summary of everything it accomplished.

- ✓ **Files changed**
- ⌚ **Total runtime**
- + **Lines of code added/changed/removed**
- 🌿 **Option to create branch and commit message**

You can click **Create branch** to push the changes. Note that:

- You are the branch owner
- Jules appears as the commit author
- You can open a PR from this branch in GitHub

Giving feedback mid-task

[Section titled "Giving feedback mid-task"](#)

You can send feedback to Jules while it's working:

- Type directly into the chat box
- Ask Jules to change its approach, revise code, or clarify logic
- Jules will respond and, if needed, replan or revise the task

You can intervene at any time, you're in control.

Pausing Jules

[Section titled "Pausing Jules"](#)

You can pause Jules at any time by clicking the **"pause"**.

When Jules is paused it won't do any work, and will wait for your next set of instructions. You can prompt it again, unpause it, or delete the task.

Starting tasks from GitHub Issues

[Section titled "Starting tasks from GitHub Issues"](#)

You can start a task from a **GitHub issue** by applying the label "jules" (case insensitive). Make sure that the Jules GitHub app is authorized to access the repo.

Select an issue, then click the gear icon next to "Labels". Then add the label "jules" (case insensitive) to the issue.

Soon, you will see Jules comment automatically on the issue. When Jules is finished with the issue, it will provide a link to the pull request where you can review its work.

Scheduled Tasks

Fuente: <https://jules.google/docs/scheduled-tasks/>

Scheduled Tasks

Jules allows you to set up recurring tasks ensuring continuous execution of routine codebase maintenance, monitoring, or updates without the need for manual re-prompting.

Creating a Scheduled Task

[Section titled "Creating a Scheduled Task"](#)

You can convert any standard task into a recurring one directly from the task input field.

1. Navigate to the main **Task Input** field.
2. Click the **Planning** dropdown located at the bottom right of the input area.
3. Select **Scheduled Task** from the menu.
4. Configure the **Frequency** and **Cadence** (e.g., Daily, Weekly) using the settings that appear at the top of the input field.
5. Write your task prompt and click **Submit**.

The task is now active and will execute automatically at the specified times.

Managing Scheduled Tasks

[Section titled "Managing Scheduled Tasks"](#)

Once a task is created, you can view and manage it within your dashboard.

To find your tasks, look for the **Scheduled** tab located immediately **under the text field** in the dashboard. From this view, you can:

- **Monitor Status:** See when the task last ran and when it is scheduled to run next.
- **Delete:** Remove the scheduled task permanently.

Editing an existing scheduled task is not currently supported. You will need to delete the task and create a new one.

Common Use Cases

[Section titled "Common Use Cases"](#)

Scheduled tasks are best used for maintenance jobs that require consistency but minimal human oversight. We've added three templated scheduled tasks that our friends at [Stitch](#) (Google's AI Design tool) uses. They include:

Use Case	Description
Performance	A performance-obsessed agent who makes the codebase faster, one optimization at a time.
Design	UX-focused agent who adds small touches of delight and accessibility to the user interface.
Security	A security-focused agent who protects the codebase from vulnerabilities and security risks.

Suggested Tasks

Fuente: <https://jules.google/docs/suggested-tasks/>

Suggested Tasks

Jules can autonomously scan your codebase to identify areas for improvement and propose implementation plans. These are presented as **Suggested Tasks**.

Currently, this experimental feature focuses on identifying and resolving **#TODO** comments left in your code, with more capabilities coming soon.

Enabling Suggested Tasks

[Section titled "Enabling Suggested Tasks"](#)

To receive suggestions, you must explicitly enable "Proactivity" for each repository you want Jules to monitor.

1. Navigate to the **Codebases** field in the left panel.
2. Select the specific repository (codebase) you wish to activate.
3. With the codebase selected, open the **Proactivity** tab in the left panel.
4. Toggle **Proactivity** to **On**.

Once enabled, Jules will immediately begin scanning the codebase. After a few minutes, if a resolvable issue is found, a suggestion will appear for your review.

Note: At launch, you are only able to enable proactivity on up to 5 repositories.

Reviewing and Running Suggestions

[Section titled "Reviewing and Running Suggestions"](#)

When Jules identifies a potential task (such as a simple `#TODO`), it will surface a suggested fix. You will also receive an email periodically when new tasks have been found.

1. Review the suggestion card that appears in your dashboard.
2. Click on the suggestion to see the context and the rationale.
3. If you agree with the plan, click **Start** to have Jules start working on the task.

Feedback and Dismissal

[Section titled "Feedback and Dismissal"](#)

You remain in control of your backlog. If a suggestion is not relevant or not a priority, you can dismiss a suggestion. Jules uses this feedback to better understand your preferences and improve future suggestions.

Configuration & Limits

[Section titled "Configuration & Limits"](#)

- **Scan Frequency:** Jules scans your repositories periodically to look for new suggestions.
- **Repository Limit:** You can currently enable Suggested Tasks on up to **five repositories**.
- **Current Scope:** The feature currently looks for **#TODO comments** that describe resolvable tasks.

Reviewing plans & giving feedback

Fuente: <https://jules.google/docs/review-plan/>

Reviewing plans & giving feedback

Once you start a task in Jules, it generates a **plan** before writing any code. This lets you know the direction Jules will take while it works on the task. From here, you can also iterate with Jules on the plan before any code is written.

Reviewing the plan

[Section titled "Reviewing the plan"](#)

After setup (cloning your repo, initializing the VM, and installing dependencies), Jules will present its plan.

You'll see:

- A natural language description of what Jules intends to direction
 - Step-by-step breakdowns
 - Any assumptions or setup steps
1. Open each step by expanding them to read through each one.
 2. Give feedback using the chat input if you need.
 3. Click **approve plan** when you're ready for Jules to begin executing and writing code.

Note: If you navigate away, Jules will eventually auto-approve the plan, which is set on a timer. So there is no need to babysit.

Giving feedback

[Section titled "Giving feedback"](#)

At any point, you can use the chat box to talk to Jules.

- Ask it to revise a step
- Point out something it missed
- Clarify your original request
- Answer its questions, or provide additional information Jules may need

Jules will respond and update its plan if needed.

Reviewing code changes

Fuente: <https://jules.google/docs/code/>

Reviewing code changes

Once you approve a plan, Jules begins working inside a virtual machine. As it completes steps, you'll see an activity log with Jules' thoughts and code updates. This page walks you through what to look for and how to respond.

Activity feed

[Section titled "Activity feed"](#)

As Jules works, you'll see a real-time **activity feed** that logs:

- Each step it completes
- Descriptions of what it did
- Any outputs or errors encountered
- Requests for additional information or feedback

This feed gives you insight into Jules' decision-making and progress.

Code diffs

[Section titled "Code diffs"](#)

When Jules changes code, you'll see a **mini diff** directly in the feed. For a more complete view:

- On the right pane, you can view a full screen expanded **diff editor** to see all changes across files
- The **diff editor** only shows files where it modified or added code.
- Expand the diff editor to full screen, or drag the left sidebar of the diff editor to slide it to your preferred width

You can download and copy code that Jules has written from the download and copy icons located in the top right of the diff editor panel. When you copy code, only the updated code will save to your clip board (not the full diff).

This is your central hub for understanding the scope of the changes Jules made to your repo.

Interactive feedback

[Section titled "Interactive feedback"](#)

You can interact with Jules in real-time through the chat box:

- Ask it to revise logic or naming
- Request additional tests or cleanup
- Give corrections like "return an empty string instead of None"

Task summary

[Section titled "Task summary"](#)

When the task completes, Jules provides a final summary which includes:

-  **Files changed**
-  **Total runtime**
-  **Lines of code added/changed**

-  **Branch name** and **commit message**

Pushing to GitHub

[Section titled "Pushing to GitHub"](#)

Click **Publish branch** or **Publish PR** to push Jules' changes to GitHub:

- Jules will appear as a commit author on the branch, if you manually create a PR from the branch, you will be the PR author
- When publishing a PR, Jules will appear as the creator of the PR

Once a branch is published, you can continue editing the branch, review it as a GitHub PR, or delete it.

Once a PR is created, you can send it for review, or merge the PR into your codebase.

Managing tasks and repos

Fuente: <https://jules.google/docs/tasks-repos/>

Managing tasks and repos

To start a new task:

1. Click the Jules icon to return to the home screen and a blank prompt input
2. Use the **+** button in the top navigation on the task view
3. From within a specific repo view, click **new task**

You can launch multiple tasks at a time to run them simultaneously. If you are viewing an existing task, or repo page when you kick off a new task, that task will load and begin running in the background. You will have to navigate to the task to check for updates, approve the plan, etc.

Each task runs in its own virtual machine and maintains its own logs, environment setup, and code changes.

Pausing or deleting tasks

[Section titled "Pausing or deleting tasks"](#)

Sometimes you need to stop Jules mid-task or clean up old runs.

- To **pause** a task: click the **pause**.
- To **delete** a task: hover over the task in the repo view and click the **trash icon**.

Paused tasks can be resumed later; deleted tasks are removed permanently.

Managing repositories

[Section titled "Managing repositories"](#)

Jules can only access repositories you explicitly allow through GitHub. In the future, Jules will work with more version control systems.

Selecting a repo

[Section titled "Selecting a repo"](#)

If you've already granted Jules access to all your repos:

- Use the dropdown when starting a task to choose any authorized repo
- Enroll new from the left sidebar
- Search for your repo using the **repo selector**

Granting access to more repos

[Section titled "Granting access to more repos"](#)

To give Jules access to more of your repositories:

1. Go to github.com
2. Click your profile photo —> **settings**
3. In the left sidebar, click **applications**
4. Find **Google Labs Jules** and click **configure**
5. Under **repository access**, select additional repos
6. Click **save**

You can also authorize new repositories via the **repo selector** in Jules. Open the repo selector and scroll to the bottom to click on **+Add repository**. This will take you to GitHub, where you can select additional repositories to grant access for Jules.

Repo view

Fuente: <https://jules.google/docs/repo/>

Repo view

The repo view gives you a focused workspace for interacting with a specific repository. From here, you can view task history, manage running tasks, and configure your environment.

Accessing the repo view

[Section titled "Accessing the repo view"](#)

There are two ways to open a repo view:

1. Click on any repository name from the sidebar task list
2. Select a repo from the task picker, then click into it after starting a task

This opens a view scoped to that repository, including task history and available actions.

Viewing tasks by repo

[Section titled "Viewing tasks by repo"](#)

Inside the repo view, you'll see a list of all tasks associated with that repo:

- **Running** tasks are listed at the top
- **Completed** tasks include full logs and diffs
- **Failed** or **waiting** tasks are clearly labeled

You can click on any task to reopen it, review the plan, or continue feedback.

Starting a new task

[Section titled "Starting a new task"](#)

From the repo view, you can start a new task scoped to that repository:

- Click **new tasks** in the top right corner
- This will preselect the repo and branch
- Enter your prompt and optimal setup script as usual

Errors and failures

Fuente: <https://jules.google/docs/errors/>

Errors and failures

Even though Jules automates many parts of the development process, things can go wrong such as environment misconfigurations to task failures. This page covers how Jules reports errors, what it does automatically, and how you can debug issues.

How errors are reported

[Section titled "How errors are reported"](#)

Jules surfaces errors in two key places:

- The **activity feed**, where the step failed is logged
- A **notification badge** (red dot) in the UI

These errors can show up whether the task has failed completely or simply requires your intervention.

Automatic retry behavior

[Section titled "Automatic retry behavior"](#)

Jules will attempt to retry failed steps automatically when possible. For example:

- Network hiccups
- Transient install errors
- Slow dependency resolutions

After multiple retries, if the problem persists, the task will be marked as **failed**.

Common causes of failure

[Section titled "Common causes of failure"](#)

The most frequent issues are:

- Incomplete or missing **environment setup scripts**
- Prompts that are too vague or overly broad
- Repos with unusual or nonstandard build systems
- Long-running processes (like `npm run dev`) included in setup

Debugging environment setup

[Section titled "Debugging environment setup"](#)

You can retry a task by:

- Clicking **rerun** from the task summary view
- Modifying your setup script or prompt before restarting the task

Make sure to address any specific feedback from the failure logs.

Limits and Plans

Fuente: <https://jules.google/docs/usage-limits/>

Limits and Plans

Jules Plans

[Section titled "Jules Plans"](#)

Jules is available with different plans designed to meet your specific needs, from individual exploration to intensive, professional development. For our initial launch, paid plans are accessed through a Google One subscription.

Note on Plan Availability

Paid Jules plans are accessed via a Google AI Plans subscription, which is currently available only for individual Google Accounts (ending in @gmail.com).

We are actively working on providing upgrade paths for other user types. In the meantime, If you are a business power user and need more access to Jules, please fill out [this interest form](#) and we will get back to you.

Plan Comparison

[Section titled "Plan Comparison"](#)

The table below outlines the key differences between the available plans. All plans have access to the same powerful Gemini models, with different number of task limits. These offerings will evolve, with more advanced models and features planned in the future.

Plan	Jules	Jules in Pro	Jules in Ultra
Best For	Evaluating Jules on real work	Daily coding with higher intensity	Power users & agent-heavy workflows
Daily Tasks (rolling 24 hours)	15	100	300
Concurrent Tasks	3	15	60
Model Access	Gemini 2.5 Pro	Higher access to the latest model (starting with Gemini 3 Pro)	Priority access to the latest model (starting with Gemini 3 Pro)

How to Upgrade

[Section titled “How to Upgrade”](#)

You can access the powerful features of our paid plans by subscribing to a corresponding Google AI plan.

Jules in Pro

[Section titled “Jules in Pro”](#)

Jules in Pro is included as a benefit of the [Google AI Pro plan](#). Upgrading gives you significantly higher task limits, perfect for professional development workflows.

Jules in Ultra

[Section titled “Jules in Ultra”](#)

Jules in Ultra is included as a benefit of the [Google AI Ultra plan](#). This plan provides our highest limits for the most demanding, large-scale projects and workflows.

Frequently Asked Questions

[Section titled “Frequently Asked Questions”](#)

Why are paid plans only available for @gmail.com accounts right now?

For our initial launch, we are launching in the Google AI Plans, which currently serves individual Google Accounts. We are working to quickly support other upgrade paths for users who have enterprise or Workspace accounts.

Will the features and limits in these plans change over time?

We may adjust limits and features as we learn how people are using the product. Our goal is to continue adding value, especially to the Pro and Ultra tiers, with differentiators like access to more advanced models, queue prioritization, more planned for the future.

If my Google One plan is shared with my family, are my Jules task limits shared?

For this launch, task limits are not shared or pooled. Each member of a shared family plan is treated as an individual user with their own set of task limits.

What happens if I use all my tasks for the day?

Once you reach your daily task limit (rolling 24 hour window), you will not be able to trigger new tasks until the limit resets. When you are at your limit, you may see calls to action within the product prompting you to upgrade to a higher plan.

I have a Google AI plan, but I can't access Jules. Why?

Jules has a stricter age requirement than some Google One plans. To use Jules, you must be 18 years of age or older. Therefore, some members of a Google One plan who are under 18 may not have access to Jules benefits, even if the plan itself is active

Can I still access my tasks when I hit a limit?

When you hit a limit:

- The **new task** button will be disabled
- A tooltip or error message will explain the reason
- You can continue reviewing or managing existing tasks
- Task history and feedback are unaffected

Feedback and support

Fuente: <https://jules.google/docs/feedback/>

Feedback and support

Jules is in active development, and your feedback directly shapes what we build next. We want to hear from you whether you've found a bug, want to suggest an improvement, or have an idea for a feature.

Giving feedback in the UI

[Section titled "Giving feedback in the UI"](#)

You can send feedback directly from within Jules:

1. Click the **feedback** button in top the navigation bar
2. Select 👍 or 👎
3. Fill out the short form and submit

Your feedback goes directly to the Google Labs team.

What happens after you submit feedback

[Section titled "What happens after you submit feedback"](#)

Your feedback is reviewed by the Jules product and engineering team regularly. We use it to:

- Prioritize new features and fixes
- Understand friction points in your workflow
- Identify common patterns and corner cases

We won't be able to reply to every submission, but we do read all of them.

Feature requests

[Section titled "Feature requests"](#)

Got something you'd love Jules to support? Tell us:

- What you were trying to do
- What got in your way
- How you imagine it could work

Clear, real-world examples are especially helpful.

Thank you for helping us make Jules better.

FAQ

Fuente: <https://jules.google/docs/faq/>

FAQ

What is Jules?

[Section titled "What is Jules?"](#)

Jules is a software coding agent that helps you fix bugs, add documentation, update your app, and implement new features. It integrates with GitHub and works autonomously — meaning you can submit a task, go do something else, and return when it's done. Jules is currently in Public Beta.

Is Jules available without cost?

[Section titled "Is Jules available without cost?"](#)

Jules has multiple plans to fit your development needs, including a plan available to all users without cost. See the [Limits and Plans](#) section for more details.

How does Jules work under the hood?

[Section titled "How does Jules work under the hood?"](#)

Each task runs in a fresh virtual machine where Jules clones your repo, installs dependencies, and makes changes based on your prompt. You can provide setup scripts to ensure your project builds and tests correctly.

How does Jules run code, and what should I know about security?

[Section titled "How does Jules run code, and what should I know about security?"](#)

When you run code in Jules, it's executed in a secure, cloud-based virtual machine (VM) with internet access. While this gives you powerful tools to test, build, and debug in context, it's important to treat the environment with the same security precautions you would for any public or shared compute surface. If you're not sure whether something is safe to run, we recommend reviewing it carefully (including non-code components). Jules is a large language model based system which operates on both the code and non-code files in a repository.

You are responsible for the code you run. That means:

- **Don't commit secrets** (like API keys, tokens, or credentials) to your repo, especially if you're pulling code into Jules from a Git provider.
- **Avoid known security vulnerabilities** in your dependencies or scripts. Consider following [GitHub's Quickstart](#) for securing your repository if you're using GitHub.
- **Be cautious with third-party packages or shell commands** that could compromise your system, your data, or others.

Can Jules run long-lived commands like `npm run dev`?

[Section titled "Can Jules run long-lived commands like npm run dev?"](#)

No. Long-running processes like dev servers or watch scripts aren't currently supported in setup scripts. Use discrete install/test commands instead.

What languages does Jules support?

[Section titled "What languages does Jules support?"](#)

Jules is language agnostic but works best with projects that use:

- JavaScript/TypeScript
- Python
- Go
- Java
- Rust

Support depends on what's installed on the VM and the clarity of your environment setup script.

Can I leave Jules while its working?

[Section titled "Can I leave Jules while its working?"](#)

Yes! Jules is designed to be autonomous. You can leave the app after submitting a task. Make sure to [enable notifications](#) so you'll be alerted when a plan is ready or a task completes.

How do I provide feedback or report a bug?

[Section titled "How do I provide feedback or report a bug?"](#)

Click the [feedback](#) button in the bottom left of the Jules UI. No account or tracking system required — feedback goes straight to the team.

What happens if a task fails?

[Section titled "What happens if a task fails?"](#)

Jules will retry automatically. If it continues to fail, it will mark the task as failed and notify you. Common causes include broken setup scripts or vague prompts.

You can revise and rerun the task once you've addressed the issue.

How many tasks can I run?

[Section titled "How many tasks can I run?"](#)

Limits are listed [here](#).

Can I change which repos Jules has access to?

[Section titled "Can I change which repos Jules has access to?"](#)

Yes. Go to your GitHub settings:

1. Click your profile —> **settings**
2. Select **applications** in the sidebar
3. Find **Google Labs Jules** and click **configure**
4. Adjust repository access

Then refresh Jules and your new repos will appear. [Learn more about managing tasks and repositories](#).

Does Jules train on private repos?

[Section titled "Does Jules train on private repos?"](#)

No. Jules does **not** train on private repository content. Privacy is a core principle for Jules, and we do not use your private repositories to train models. [Learn more](#) about how your data is used to improve Jules.

Contact

Fuente: <https://jules.google/docs/contact/>

Contact

If you have any questions, feedback, or need assistance, please join our [Discord](#) or follow us on [X](#).

Documentación de Jules Agent - Módulo 1: Guías y Documentación General